Correspondencia Objeto-Relacional con Hibernate

Acceso a Datos

Índice

- Objetivos
- Introducción: el desfase objeto-relacional
- Prerrequisitos para utilizar Hibernate
- Elementos y ficheros de Hibernate
- Ciclo de vida de los objetos
- Correspondencia en las relaciones
- Lenguaje de Consulta HQL
- Bibliografía

Objetivos

- **Conocer el propósito y comprender los fundamentos de la correspondencia objeto-relacional (en inglés, ORM).**
- **Conocer y familiarizarse con Hibernate, una de las herramientas y framework más utilizados para el ORM.**
- **Entender y probar la configuración de Hibernate, y la representación en éste de las relaciones de las BD relacionales.**
- **Crear clases de Java para ser usadas con Hibernate y establecer la correspondencia con la BD.**
- **Conocer y utilizar HQL, el lenguaje de consulta de Hibernate.**

Introducción: el desfase objeto-relacional

- **Cuando se usa un programa orientado a objetos y una base de datos relacional, es fácil ver que los datos en un sistema y el otro no son los mismos y que se necesita una traducción de uno a otro.**
- **A esto se le llama el <u>desfase objeto-relacional.</u>**
- **El problema no es trivial, ya que además del esfuerzo en la conversión, hay casos más complejos que otros.**
  - **Por ejemplo, establecer correspondencia entre objetos con referencias a otros objetos, las relaciones entre tablas (1:1, 1:n y N:M), subclases de objetos…**

Introducción: el desfase objeto-relacional

- **Técnicas para solucionar el desfase objeto-relacional:**
  - **Bases de datos de objetos: bases de datos que almacenan directamente objetos. En la actualidad, tienen un uso muy reducido**
  - **Bases de datos objeto-relacionales: bases de datos relacionales con capacidad de gestionar objetos en concreto. Se usan en algunos nichos de mercado.**
  - **Correspondencia objeto-relacional (ORM): establece una correspondencia entre las clases de un lenguaje OO y las tablas de una base de datos relacional, además de proporcionar mecanismos para el registro de los datos en la BD y la recuperación de los objetos con la información en tablas.**

Introducción: Hibernate

- **Hibernate es un framework de ORM para Java con herramientas para su utilización.**
- **Su API implementa ademas la Api de JPA (Java Persistence API) a partir de la versión 2.0, haciendo que se posible utilizarlo en aplicaciones y entornos que utilicen JPA.**
- **Trabaja con objetos transitorios y persistentes a la vez, creando su correspondencia directamente, asociados a una sesión de conexión con la BD, y pudiendo trabajar mediante transacciones.**
- **Dispone de un lenguaje de consulta (HQL) basado en SQL y con sintaxis similar, pero independiente una versión de SQL concreta, pudiendo traducirse a varios de los habituales.**

Prerrequisitos para utilizar Hibernate

Hibernate hace mapeo entre una base de datos y objetos de Java declarados mediantes clases POJO.

- **POJO: del ingles, Plain Old Java Object, o lo que es lo mismo, las clases normales que declaran los desarrolladores para representar sus objetos. No son parte ni descendientes de frameworks o APIs**

Sin embargo, no todas las correspondencias son posibles. Para poder automatizarse, los datos de ambas partes tienen que seguir una serie de reglas.

Si los datos no cumplen con las siguientes reglas, deben haber un proceso de adaptación antes de utilizar hibernate.

Prerrequisitos para utilizar Hibernate

Del lado de la BD:

- **Las tablas tienen que tener clave primaria.**
- **La clave primaria no puede cambiar (si son varias columnas, afecta a todos los valores).**

Es fácil ver que para automatizar la correspondencia se necesita una clave primaria inmutable.

Una forma sencilla de cumplir con esta regla, facilitar la correspondencia y aumentar la eficacia del sistema es crear una clave primaria numeria autogenerada.

Prerrequisitos para utilizar Hibernate

Del lado de los objetos de Java:

- **Las clases deben implementar la interfaz Serializable (con incluir implements Serializable en la definición basta).**
- **La clase debe contar con un constructor por defecto.**
- **Debe cumplir con las convenciones de getters y setters por cada campo que sea correspondido.**
  - **Recordad que la convención es que si tengo el atributo “abc” su getter es getAbcy su setter es setAbc**

Elementos para usar Hibernate

Si un proyecto quiere utilizar Hibernate como ORM, además de las librerías externas de Hibernate (dependiendo de la versión, unas 10 librerías), debe incluir algunos ficheros de configuración.

Hibernate.cfg.xml

Fichero de configuración básico para hibernate. Contiene:

- Configuración básica de conexión
- Opciones de control
- Indicaciones básicas de mapeo
- Linkados a otros ficheros de configuración

Elementos para usar Hibernate

Hibernate.cfg.xml (ejemplo):

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
                                         "http://www.hibernate.org/dtd/hibernate-configuration-3.0
<hibernate-configuration>
 <session-factory name="">
  <property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
  <property name="hibernate.connection.password">asdf.1234</property>
  <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/hola_hibernate</property>
  <property name="hibernate.connection.username">root</property>
  <property name="hibernate.default_schema">hola_hibernate</property>
  <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
  <property name="hibernate.show_sql">true</property>
  <property name="hibernate.format_sql">true</property>
  <mapping class="ObjetosBasicos.Personajes"/>
 </session-factory>
</hibernate-configuration>
```

Elementos para usar Hibernate

Clases POJO con definiciones de JPA:

Una de las técnicas de automatización de la correspondencia es incluir en la definición de la clase anotaciones de JPA para indicar la tabla y la columna a la que hacen referencia la clase y los atributos descritos, respectivamente.

Aunque son muchas las anotaciones y configuraciones posibles en JPA, a continuación se muestra una lista de los más frecuentes y útiles para empezar.

Elementos para usar Hibernate

Anotaciones JPA importantes:

- **@Entity: Indica que la clase es una entidad que se va a mapear con una tabla de la base de datos. Los campos de la clase se mapean con columnas de la base de datos.**
- **@Table: indica que una entidad se mapea con la tabla indicada. Por defecto el nombre de la tabla será el nombre de la clase Java.**
- **@Id: indica que el campo anotado va a ser el identificador de la entidad. Suele ser un Long id. La columna con la que se mapea en la base de datos es la clave primaria de la tabla.**
- **@GeneratedValue: El identificador se genera automáticamente por parte de la base de datos cuando la entidad se hace persistente.**
- **@Column: Todos los atributos se mapean con campos de la tabla. En el caso de no utilizar la anotación @Column se mapea con un campo con el mismo nombre que el atributo. Sirve para indicar características del esquema de la columna en la que se mapea el campo. Por ejemplo, puede llevar el modificador NOT NULL.**

Elementos para usar Hibernate

Anotaciones JPA importantes (Ejemplo):

```
@Entity
@Table(name="personajes")
public class Personajes implements java.io.Serializable {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Integer id;
    @Column
    private String nombre;
```

TUTORIAL:
https://www.tutorialspoint.com/hibernate/hibernate_annotations.htm

Elementos para usar Hibernate

<u>Fichero de ingeniería inversa (hiberante.reveng.xml):</u>

Fichero de especificación de las tablas a utilizar para las tablas a utilizar en las correspondencia.

Es un fichero básico en la automatización de procesos con las herramientas de Hibernate

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-reverse-engineering PUBLIC "-//Hibernate/Hibernate Re

<hibernate-reverse-engineering>
  <table-filter match-catalog="hola_hibernate" match-name="personajes"/>
</hibernate-reverse-engineering>
```

Elementos para usar Hibernate

<u>Ficheros de configuración hbm.xml:</u>

Alternativamente a las anotaciones JPA se puede utilizar ficheros de configuración xml.

Contienen una información similar a las anotaciones, con la diferencia de que dicha información no está directamente en la definición de la clase, si no que está en un recurso aparte.

Para utilizar estos ficheros, también hay que linkarlos en el hibernate.cfg.xml

Elementos para usar Hibernate

<u>Gestión de sesiones y transacciones:</u>

Usando Hibernate, las conexiones a la BD para hacer la correspondencia se hacen en sesiones de trabajo, y éstas a su vez agrupan operaciones mediante transacciones.

La API de Hibernate ofrece un SesionFactory para crear y gestionar las conexiones.

La forma habitual de utilizarlo es utilizándolo con un patrón Singleton.

Una estrategia frecuente es centralizar la creación y uso de la factoría y de las sesiones mediante una clase gestora, llamada normalmente HibernateUtil.java.

Elementos para usar Hibernate

Ejemplo de creación de SessionFactory en HibernateUtil.java:

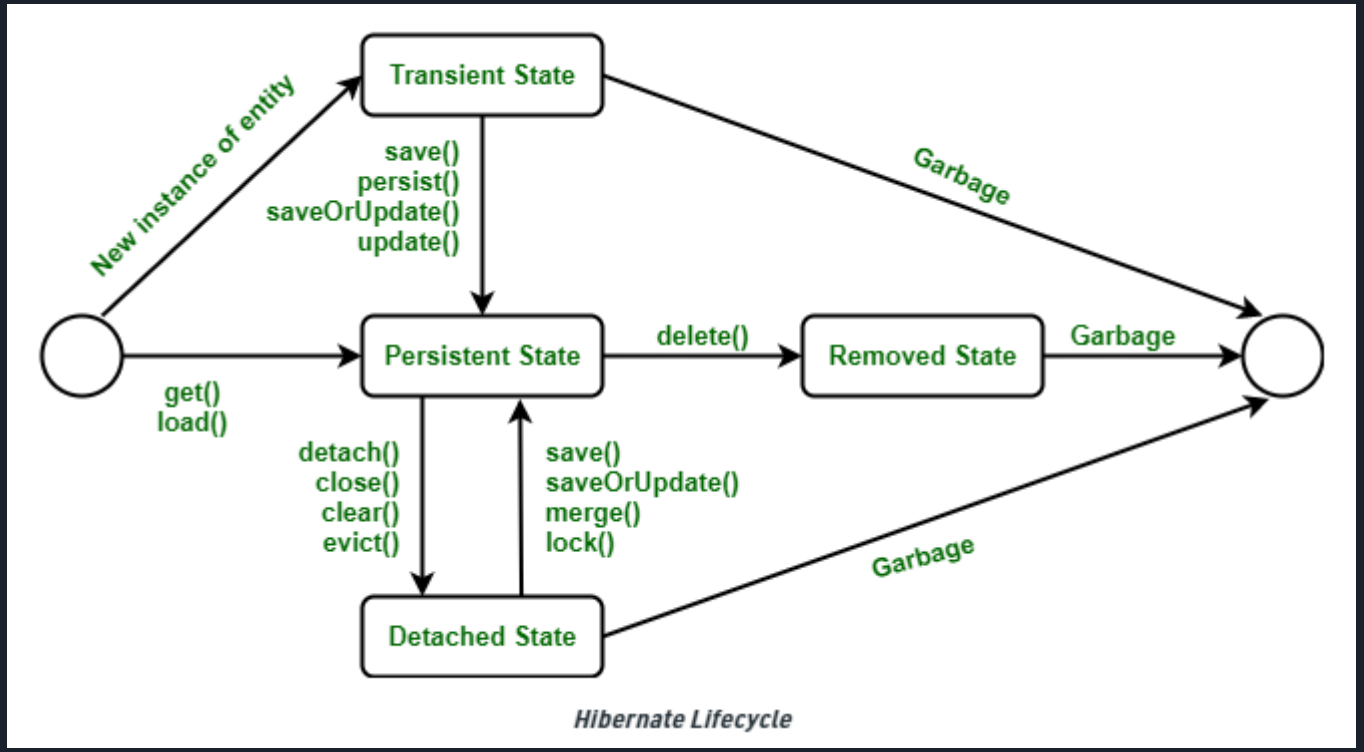
```java
/**
 * Crea la factoria de sesiones
 */
public static void buildSessionFactory() {
    Configuration configuration = new Configuration();
    configuration.configure();
    // Se registran las clases que hay que mapear con cada tabla de la base de datos
    // Necesario con la estrategia de las clases con anotaciones
    configuration.addAnnotatedClass(Personajes.class);

    ServiceRegistry serviceRegistry = new StandardServiceRegistryBuilder()
            .applySettings(configuration.getProperties()).build();
    sessionFactory = configuration.buildSessionFactory(serviceRegistry);
}
```

El ciclo de vida de los objetos

En hibernate se consideran que los objetos de Java pueden estar en cuatro estados fundamentales:

- **<u>Transitorio:</u> cuando el objeto solo existe en la aplicación de Java y se perderá cuando se pierdan sus referencias.**
- **<u>Persistente o manejado</u>: cuando se ha linkado con la BD utilizando el sistema de correspondencia.**
  - **Importante: en este estado, los cambios que se hagan sobre el objeto Java se verán reflejados en la BD al acabar la transacción.**
- **<u>Separado:</u> un objeto que estaba siendo manejado deja de serlo, perdiéndose todo cambio nuevo que se haga.**
- **<u>Eliminado:</u> Estado donde se ha eliminado el objeto persistente correspondiente, por lo que el objeto en java ya está listo para ser liberado.**

El ciclo de vida de los objetos



Correspondencia en las relaciones

Tipos de relaciones:

- 1 a 1 (uno a uno)
  - @OneToOne(cascade = CascadeType.TIPO_CASCADA)
    - Aplica a un atributo que represente un objeto de otro tipo.
  - @JoinColumn(name = "nombre_columna")
- 1 a N (uno a muchos)
  - @ManyToOne
    - Aplica a un atributo que represente un objeto de otro tipo.
  - @OneToMany (normalmente para indicar relación contraria).
    - Aplica a una colección de objetos del otro tipo.
  - @JoinColumn(name = "nombre_columna")

Correspondencia en las relaciones

Tipos de relaciones:

- M a N (muchos a muchos):
  - @ManyToMany
    - Aplica a una colección de objetos del otro tipo.
  - @JoinTable(name = "tabla_relacion",
    - joinColumns = { @JoinColumn(name = "nombre_columna_miID") },
    - inverseJoinColumns = {@JoinColumn(name = "nombre_columna_IDOtro") }

Correspondencia en las relaciones

Otras categorías de tipos

- Unidireccionales: los datos solo se tienen en un objeto.
- Bidireccionales: los datos de la relación se tienen en los dos tipos de objetos
  - Parámetro mappedBy= nombreAtributo que tiene la relación inversa
  - A ese atributo se le llama también el dueño de la relación, porque es quien define la relación.

- Funciones helper en caso bidireccional:
  - Por completitud, será necesario que cuando haya un cambio en un objeto de la relación, se produzca el cambio equivalente en el otro lado de la relación.
  - Se suelen añadir funciones para añadir y quitar los datos correspondientes, ya sea anulando o asignado el dato del atributo unitario (en el caso extremos “1” de la relación), o añadiendo y quitando de la lista en caso de relaciones (en el caso de extremos “N” de la relación).

Correspondencia en las relaciones

- Tipos de Cascade más frecuentes:
  - CascadeType.Persist
    - Cuando un objeto se persiste, se persisten sus objetos relacionados.
  - CascadeType.Remove
    - Cuando un objeto se borra, se borran sus objetos relacionados.
  - CascadeType.ALL
    - Todo acción hecha para un objeto se replica a sus objetos relacionados
- orphanRemoval:
  - Elimina elementos “huérfanas” de relación, es decir, sin relación.
  - Puede pasar porque se elimine la otra parte de la relación, o porque la otra parte cambie su objeto relacionado.
  - Solo puede darse en extremos “1” de relaciones (@OneToOne, @OneToMany)

Lenguaje de Consulta HQL

- Hibernate utiliza un lenguaje propio inspirado en SQL y de sintaxis similar a este para implementar las sentencias CRUD tipicas (SELECT, UPDATE…)

- Al subconjunto de las instruciones de HQL que sirven para las especificaciones de JPA se les llama JPQL. Son especialmente útiles porque nos abstraen de la BD que se esté utilizando, pudiendo cambiar en el futuro sin que cambie el programa.

- Por esto último, el cambio mayor a tener en cuenta sobre HQL es el uso de clases y notación de Java en vez de las tablas implementadas

Lenguaje de Consulta HQL

- La consultas se realizan mediante objetos tipo Query, análogo a la interfaz PreparedStatement de JDBC.
- Estos objetos se crean mediante la sesión, y si las sentencias ejecutan cambios, deberá iniciarse y hacerse el commit de una transacción (una vez más, como en JDBC).
- La creación de un objeto tipo Query se hace mediante el siguiente método:

```
sesion.createQuery("String con la consulta")
```

  - Donde el String está en formato HQL con placeholders.
  - En vez de nombres de tablas se usan las clases Java
  - Se pueden utilizar objetos dentro de objetos de java (objeto.atributo). Internamente, Hibernate lo traducirá a los JOINS necesarios.
  - Los placeholders en HQL se representan con :nombreVariable y se pueden utilizar nombrandolos, en vez de utilizar su posición numérica (aunque este sistema también es válido).

Lenguaje de Consulta HQL

- Los métodos más interesantes de los objetos Query son:
  - getResultList():
    - Devuelve una lista de resultados de la consulta. Si la consulta se hace para recibir objetos de Java, serán objetos Java manejados por Hibernate
  - getSingleResult()
    - En caso de saber que el resultado es un único elemento, se puede utilizar este sistema en vez de la lista para tener mayor facilidad de uso
  - getFirstResult()
    - Solo obtiene el primer resultado de la consulta, asegurando el resultado único.
  - setParameter(int pos, Object value) / setParameter (String nombre, Object value)
    - Setea el valor de un parametro, pudiendo utilizarse por posicion o por nombre del campo
    - El valor puede ser una lista de valores (para utilizar en casos WHERE x IN :listaValores)
  - SetFirstResult(int pos) / setMaxResults(int maxResult)
    - Setea el valor inicial y al valor máximo de la consulta
    - Muy útil para establecer límites (ejemplo, máximo 1 y asegurar el resultado único), o empezar en una posición concreta, muy útil para implementar paginación.

Lenguaje de Consulta HQL: Ejemplos

- Ejemplos de lectura de tabla.
- Observar que no se utiliza una tabla, si no la clase Personaje (En el ejemplo, la tabla es personajes).
- Se puede utilizar o no la parte de SELECT. Devuelve lista del tipo Personaje.

```java
//consulta basica de todo
System.out.println("Consulta 1");
Query query = s.createQuery("FROM Personaje");
List<Personaje> personajesDisponibles = query.getResultList();
for(Personaje p: personajesDisponibles) {
    System.out.println(p.getNombre());
}

//consulta basica de todo (segunda forma)
System.out.println("Consulta 2");
Query query2 = s.createQuery("Select per FROM Personaje per");
List<Personaje> personajesDisponibles2 = query2.getResultList();
for(Personaje p: personajesDisponibles2) {
    System.out.println(p.getNombre()+" "+p.getPuntos());
}
```

Lenguaje de Consulta HQL: Ejemplos

- Se puede utilizar la consulta para devolver datos o columnas en concreto, pero en ese caso se devuelve una lista de Object, ya que no hay datos suficientes para formar el objeto.
- Similar a los datos de un ResultSet de JDBC.

```
//consulta basica por columnas
System.out.println("Consulta 3");
Query query3 = s.createQuery("Select p.nombre, p.puntos  FROM Personaje p");
List<Object[]> resultados = query3.getResultList();
for(Object[] lista: resultados) {
    System.out.println(lista[0]+" "+lista[1]);
}
```

Lenguaje de Consulta HQL: Ejemplos

- Se puede utilizar la consulta para devolver datos o columnas en concreto, pero en ese caso se devuelve una lista de Object, ya que no hay datos suficientes para formar el objeto.
- Similar a los datos de un ResultSet de JDBC.

```
//consulta basica por columnas
System.out.println("Consulta 3");
Query query3 = s.createQuery("Select p.nombre, p.puntos  FROM Personaje p");
List<Object[]> resultados = query3.getResultList();
for(Object[] lista: resultados) {
    System.out.println(lista[0]+" "+lista[1]);
}
```

Lenguaje de Consulta HQL: Ejemplos

- Consultas con parámetros para filtrar (sencillos y con listas)

```
//consulta filtrada
System.out.println("Consulta 4");
Query query4 = s.createQuery("Select p FROM Personaje p WHERE nombre is :unNombre"+
        " AND puntos > :minimoPuntos");
query4.setParameter("unNombre", "MegaMan");
query4.setParameter("minimoPuntos", 20);
List<Personaje> personajesMegaman = query4.getResultList();
for(Personaje p: personajesMegaman) {
    System.out.println(p.getNombre()+" "+p.getPuntos());
}

//consulta filtrada mediante una lista
System.out.println("Consulta 5");
Query query5 = s.createQuery("Select p FROM Personaje p WHERE nombre IN (:unaLista)" );
String[] listaNombre = {"Superman", "Spiderman"};
List<String> laLista = Arrays.asList(listaNombre);
query5.setParameter("unaLista", laLista);
List<Personaje> personajesNoMegaman = query5.getResultList();
for(Personaje p: personajesNoMegaman) {
    System.out.println(p.getNombre()+" "+p.getPuntos());
}
```

Lenguaje de Consulta HQL: Ejemplos

- Consultas con máximo 1 para asegurar el resultado único.
- IMPORTANTE: observar que no se hacen JOINS, pero se navega a través de lo que internamente son varias tablas utilizando los atributos de los objetos de java:
  - objeto.atributoObjeto.otroAtributoObjeto

```
//consulta de valor Unico
System.out.println("Consulta 7");
Query query7 = s.createQuery("SELECT h.personaje.arma FROM Habilidad h WHERE h.nombre LIKE :unNombre" );
query7.setParameter("unNombre", "rayo laser");
query7.setMaxResults(1);
Arma armaAsociada = (Arma) query7.getSingleResult();
System.out.println(armaAsociada.getNombre());
```

Lenguaje de Consulta HQL: Otros casos

- Se pueden hacer consultas con SQL directamente, mediante createSQLQuery.
  - Ventaja: es el poder utilizar las capacidades particulares de un SGBD directamente.
  - Desventajas: Nos saltamos la capa de hibernate, no pudiendo utilizar objetos

```
//sentencia SQL
System.out.println("SQL directo");
Query queryDirecta = s.createSQLQuery("SELECT * FROM personajes WHERE nombre = :unNombre");
queryDirecta.setParameter("unNombre", "Megaman");

//ESto no funciona con SQL Directo
List<Personaje> listaMegaman = queryDirecta.getResultList();
for(Personaje p: listaMegaman) {
    System.out.println(p.getNombre());
}

//esta es la forma
List<Object[]> listaResultados = queryDirecta.getResultList();
for(Object[] lista: listaResultados) {
    System.out.println(lista[0]+" "+lista[1]);
}
```

Lenguaje de Consulta HQL: Otros casos

- Actualizaciones y borrados mediante consultas:
  - Se pueden utilizar las consultas de HQL para realizar UPDATE y DELETE.
  - Como se puede deducir, para cambios en objetos concreto merece más la pena utilizar los objetos Java (Recordar ciclo de vida, y operaciones get, update, remove…)
  - Las actualizaciones por consulta son especialmente útiles para hacer cambios masivos que de otra forma serían más costoso.
  - Hay que utilizar una transacción para ello.
  - Se utiliza el método executeUpdate(), y análogamente a JDBC, devuelve el número

```
//actualizacion
System.out.println("Actualizacion");
t= s.beginTransaction(); //meter transaccion
Query queryActualizacion = s.createQuery("UPDATE Personaje SET puntos = :nuevaPuntuacion WHERE nombre is :unNombre");
queryActualizacion.setParameter("unNombre", "Megaman");
queryActualizacion.setParameter("nuevaPuntuacion", 250);
int cambios = queryActualizacion.executeUpdate();
System.out.println("cambios realizados "+cambios);
t.commit();
```

Bibliografía

- Cortijo Bon, C. A, Acceso a Datos

Recursos online:

- Página oficial de Hibernate:
  - https://hibernate.org/
- Tutorial general sobre Hibernate:
  - https://www.tutorialspoint.com/hibernate/index.htm
- Tutoriales de configuración básica de Hibernate en Eclipse:
  - https://examples.javacodegeeks.com/enterprise-java/hibernate/hibernate-jpa-dao-example/
  - http://www.programandoapasitos.com/2016/01/instalacion-y-configuracion-de.html
- El ciclo de vida de los objetos en Hibernate:
  - https://www.geeksforgeeks.org/hibernate-lifecycle/
- Introducción a JPA:
  - https://www.oscarblancarteblog.com/tutoriales/java-persistence-api
- Tipos de CascadeType y OrphanRemoval:
  - https://www.nestoralmeida.com/cascade-en-jpa-hibernate/

BY NC SA

Autor: Manuel Barroso
mbarroso2@educa.madrid.org